

Tourism Government System — Ultimate Interview Guide

Targeted Q&A; for Reporting & Notification Microservices, Security, and UI Integration

■ Self Introduction (Tailored to your role)

Greet the interviewer, state your experience in building distributed enterprise systems using Spring Boot, Spring Cloud, React, and MySQL, and highlight your direct contribution to the secure Reporting and real-time Notification modules of the Tourism Government system.

■■ How to pitch this to the interviewer:

"Hi, I am a software engineer specializing in Java Full-stack development with a strong focus on Microservices architectures. Recently, I worked on the Tourism Government System, which is an 11-service platform. My core responsibilities were developing and securing the Notification Service and the Reporting Service, integrating them with our React frontend, implementing fault-tolerant Resilience4j circuit breakers, and ensuring secure communication across the ecosystem using API Gateways and JWT."

■ Explain your Project (Tourism Government System)

The project is a modular government management platform divided into 11 microservices (Eureka, Gateway, Config Server, User, Tourist, Heritage Site, Program, Event Booking, Compliance, Reporting, and Notification). It enables compliance officers to audit heritage sites, and tourists to book events, with instant notifications and automated reports.

■■ How to pitch this to the interviewer:

"Our project, the Tourism Government System, monitors compliance audits for national heritage sites and facilitates booking programs for tourists. Since it is built as a microservices architecture, every domain (like sites, compliance, reporting, and notifications) runs as an independent Spring Boot service with its own MySQL database. This ensures high availability, modularity, and rapid deployments."

■ What Design Patterns are used in your project?

Your project implements 7 major patterns: **API Gateway** (for centralized routing and security), **Service Discovery** (Eureka registry), **Externalized Config** (Config Server), **Database-per-Service** (isolated MySQL schemas), **API Composition** (Reporting service merging data from Feign clients), **Circuit Breaker** (Resilience4j fault protection), and the **Observer/Event-Consumer Pattern** (Notification alerts).

■■ How to pitch this to the interviewer:

"We leverage several classic microservice patterns: 1) API Gateway as a single entry point, 2) Service Discovery using Netflix Eureka, 3) Externalized Configuration using Spring Cloud Config, 4) Database-per-Service for loose coupling, 5) API Composition in our Reporting module to fetch and merge data, and 6) Circuit Breaker with Resilience4j to handle network timeouts gracefully."

■ What is the purpose of the Service Registry (Eureka)?

The Service Registry maps the shifting hostnames and ports of running microservice instances in real-time. Instead of services calling hardcoded endpoints (like localhost:9090), they register themselves under a logical service name (like REPORT-SERVICE) and Eureka dynamically routes their communication.

■■ How to pitch this to the interviewer:

"In a cloud environment, microservice instances scale up and down dynamically, causing their physical IP addresses to change constantly. The Eureka Service Registry acts as a dynamic phonebook. Every microservice registers its location on boot, allowing OpenFeign clients to look up target services by name rather than hardcoding static IPs."

■ Explain to me the code of Circuit Breakers, Sliding Windows, and properties.

Your Spring Boot code uses a declarative OpenFeign circuit breaker. By enabling the circuit breaker in properties, Spring wraps the Feign interfaces inside a Resilience4j proxy. The sliding window (10 requests) tracks failure rates; if 50% fail, the circuit trips OPEN, and calls route to a local fallback class.

```
spring.cloud.openfeign.circuitbreaker.enabled=true
resilience4j.circuitbreaker.configs.default.slidingWindowSize=10
resilience4j.circuitbreaker.configs.default.failureRateThreshold=50
resilience4j.circuitbreaker.configs.default.waitDurationInOpenState=10000
```

■■ How to pitch this to the interviewer:

"We configured a sliding window of the last 10 requests. If 50% or more fail, the circuit opens for 10 seconds. During this open state, network calls are completely bypassed, and Resilience4j instantly executes our local Feign fallback classes to protect the service thread pool."

■ Explain how you connect the backend and the frontend.

The React frontend (chennaioneFronted) communicates exclusively with the API Gateway (GatewayAPI) running on port 8383. The frontend passes a stateless JWT token inside the 'Authorization: Bearer ' HTTP header. The Gateway intercepts, validates the JWT, and forwards the request to the correct downstream microservice.

■■ How to pitch this to the interviewer:

"Our React frontend connects directly and exclusively to our API Gateway on port 8383 using Axios. The frontend stores the JWT token upon login and attaches it as a Bearer token in the request headers. The Gateway validates the token centrally, injects the user identity as headers, and handles routing to downstream services transparently."

■ TRICKY QUESTION: Eureka Server is in production. You change its port after 5 years. Will it make any difference in the application?

Yes, it will break all service-to-service communication immediately unless handled properly! Every registered microservice client points to Eureka using a static default URL (like port 8761). If Eureka's port changes, services will fail to register or query other services, leading to a complete system outage.

How to solve it in production without breaking anything:

- 1) **Load Balancer / DNS:** In production, we never hardcode ports or IPs. We point services to a domain name like `http://eureka.tourismgov.internal/eureka/`. If we change the port under the hood, we only update the Load Balancer target. The clients remain unchanged.
- 2) **Config Server:** Since we use a Centralized Config Server, we can update the property in one config file, and client services will automatically pull the updated Eureka URL without requiring code changes.

■■ How to pitch this to the interviewer:

"Yes, changing Eureka's port directly would instantly break registration and routing across all microservices since they look for Eureka on a specified port. However, in a professional production environment, we prevent this by using a DNS domain name or a Load Balancer URL rather than a hardcoded port. If we do have to change the port, we update the setting in our Central Config Server once, ensuring zero service redeployments."

■ Project workflow of your module on the UI (Reporting & Notification Dashboard)

The React UI displays a Notification Bell in the header. When clicked, it displays unread alerts. In the Reporting section, the Admin sees real-time compliance statistics, heritage site logs, and pending audits. The Dashboard fetches this dynamically by hitting `/tourismgov/v1/reports/dashboard`` via the Gateway.

■■ How to pitch this to the interviewer:

"When an Admin logs into the React dashboard, the UI makes a unified call to our Reporting Service. In the background, the Reporting Service uses Feign Clients to fetch data from the Event, Compliance, and User services. At the same time, the Notification Bell fires an Axios call to the Notification Service to fetch the latest unread alerts, updating the UI dynamically."

■ Explain MVC Architecture in your backend code (from Controller to Service)

The MVC (Model-View-Controller) structure separates concerns cleanly:

- 1) **Controller:** Handles incoming HTTP requests, validates request payloads, and maps parameters.
- 2) **Service Layer:** Contains all transaction logic, call-outs to Feign clients, and core calculations.
- 3) **Repository Layer:** Interacts with the database using Spring Data JPA.

■■ How to pitch this to the interviewer:

"We follow the MVC architecture. The Controller acts as the entry point, handling endpoint mapping and payload validation. It delegates all business operations to the Service interface, which contains the transaction rules, business calculations, and Feign client calls. The service then queries or persists data using Spring Data JPA Repositories."

■ What validations did you use? Explain the DTO layer.

We use Spring Boot Validation constraints (like @NotNull, @Size, and @Email) directly inside our Data Transfer Objects (DTOs) in the controller layer. The DTO layer ensures that we don't expose our database entity structures directly to the client API.

■■ How to pitch this to the interviewer:

"We use Data Transfer Objects (DTOs) to decouple our API requests/responses from database entity structures. We enforce strict data contracts at the controller level using annotations like @NotBlank, @Size, and @Min. If a client sends invalid data, Spring's MethodArgumentNotValidException intercepts the request and returns a clean 400 Bad Request error."

■ Why do we prefer Constructor Injection instead of Field Injection?

Field injection (using @Autowired on variables) is highly discouraged. Instead, we use **Constructor Injection** (often generated automatically using Lombok's @RequiredArgsConstructor).

■■ How to pitch this to the interviewer:

"We prefer Constructor Injection for three main reasons: 1) Immutability, as we can declare our dependencies as final, 2) Testability, as it allows us to easily inject mock dependencies during unit testing without resorting to reflection, and 3) Avoiding NullPointerExceptions at runtime because Spring enforces dependency resolution at instantiation."

■ Have you written test cases? JUnit & @Mock vs @InjectMocks.

Yes, we write unit tests using JUnit 5 and Mockito. We mock external services (like the Feign clients or database repositories) to keep tests isolated and fast.

* @Mock: Creates a mock stub of a dependency.

* @InjectMocks: Creates the actual class under test and injects all created @Mock fields into it.

■■ How to pitch this to the interviewer:

"Yes, we write robust unit tests using JUnit 5 and Mockito. We use the @Mock annotation to create stubs for our Repositories and Feign Clients. We then use @InjectMocks on our Service Implementation class, which automatically injects all mock dependencies into the service, allowing us to test its business logic in complete isolation."